anvl: Composable Code Transformations for Array Programs in R

by Sebastian Fischer, Daniel Falbel, Tomasz Kalinowski, Nikolai German, Martin Binder, and Bernd Bischl

Abstract We introduce the R package **anvl**, a code transformation framework for numerical array programs. Functions written in R against the package's array interface can be just-in-time (JIT) compiled, differentiated in reverse mode, and executed on CPUs and NVIDIA GPUs from the same source code. All transformations are based on first tracing an R function into a computational graph, an intermediate representation that is independent of the R interpreter. Arbitrary R code can therefore be used to construct a program, as it is "staged out" during tracing. Automatic differentiation is implemented as a transformation of one graph into another, and backends lower a graph to an executable. The main backend targets the OpenXLA compiler ecosystem via the companion packages **stablehlo** and **pjrt**, while an experimental second backend translates graphs to Fortran via **quickr**. A distinguishing feature is the treatment of data types: R values carry no data type of their own and are built into a program at the precision of the arrays they meet, and every function declares how it promotes its inputs. This gives users fine-grained control over numerical precision, which matters for statistical computations, without having to annotate the data they pass in from R. Except for the runtime, **anvl** and **stablehlo** are written in R, which makes them comparatively easy to extend with new primitive operations, transformations, and backends. We describe the design of the package, show how to extend it, and demonstrate it by implementing a Hamiltonian Monte Carlo sampler for Bayesian logistic regression, which we compare against Stan on CPU and GPU.

1 Introduction and related work

Most statistical and machine learning methods are, at their core, sequences of operations on vectors and arrays. In R, such methods can often be implemented efficiently by relying on vectorized operations and optimized linear algebra routines. However, there are cases where this is not sufficient: when an algorithm is inherently iterative, as many optimization and sampling algorithms are, interpreted R loops quickly become the bottleneck. The usual remedy is to rewrite the performance-critical parts in a compiled language such as C++ via **Rcpp** (Eddelbuettel and François, 2011). If one additionally wants gradients for optimization or sampling, or wants to run the computation on a GPU, the required machinery grows further.

In Python, these needs are covered by array libraries that combine hardware acceleration with automatic differentiation (AD). **PyTorch** (Paszke et al., 2019) is built around eager execution, where each operation is executed immediately, and later added the ability to compile programs. **JAX** (Bradbury et al., 2018; Frostig et al., 2018) instead describes itself as a framework for composable transformations of NumPy programs (Harris et al., 2020). A JAX program is traced into an intermediate representation, which can then be differentiated, vectorized, and JIT compiled via the XLA compiler. Although these libraries are mostly known for deep learning, they are equally useful for other numerical algorithms. For example, much of the Bayesian inference ecosystem in Python is built on top of JAX, including **NumPyro** (Phan et al., 2019), **BlackJAX** (Cabezas et al., 2024), and the JAX substrate of **TensorFlow Probability** (Dillon et al., 2017), and **PyMC** can hand its models to JAX-based samplers (Abril-Pla et al., 2023).

In R, **torch** (Falbel and Luraschi, 2025) brings the **LibTorch** library that underpins PyTorch to R without a Python dependency. Like PyTorch, it is primarily designed for eager execution. While it can trace code into TorchScript, TorchScript is primarily designed for deployment and is no longer actively developed upstream. For large neural networks, where the runtime is dominated by individually expensive operations, eager execution is rarely

Table 1: Overview of related R packages. Input is the language in which a program is written, Compilation the way it is executed.

Package	Input	Compilation	Reverse AD	GPU
anvl	R (array API)	JIT (XLA, Fortran)	yes	CUDA
torch	R (array API)	eager	yes	CUDA
tensorflow, keras3, greta	R via Python	XLA (Python)	yes	CUDA
quickr	R (subset)	Fortran	no	no
compiler	R	byte code	no	no
rstan, cmdstanr	Stan language	C++	yes	partly (OpenCL)
nimble	BUGS-like, R (subset)	C++	yes	no
TMB, RTMB	C++ / R	C++ / tape	yes	no
nabla, salad, dual	R	none	no	no

a bottleneck. For the comparatively small models that are common in statistics, however, the per-operation overhead of eager execution can dominate, which makes whole-program compilation attractive. The packages **tensorflow**, **keras3**, and **greta** (Golding, 2019) give access to XLA-compiled programs, and JAX itself can be used via **reticulate**. They do so by wrapping Python libraries, which requires managing a Python installation and converting objects between the two languages.

The lack of native GPU support in R has consequences beyond runtime. Methodological research on how to use accelerators for expensive statistical methods can hardly be carried out in R today. For example, a recent line of work on Markov chain Monte Carlo methods runs hundreds or thousands of short chains in parallel on a GPU, which requires new adaptation schemes and convergence diagnostics (Hoffman et al., 2021; Hoffman and Sountsov, 2022; Margossian et al., 2024). Such methods are developed and implemented with JAX, and R users can only use them through Python.

Various R packages provide one or two of the three capabilities of JIT compilation, AD, and GPU execution. The byte-code **compiler** included in R (R Core Team, 2025) compiles arbitrary R code, but because it has to preserve R's full semantics, the optimizations it can perform are limited (Morandat et al., 2012). The **quickr** package (Kalinowski, 2026) compiles a typed subset of R to Fortran, and **ast2ast** (Krämer and Xie, 2026) translates a subset of R to C++ and supports AD. **odin** (FitzJohn, 2025) and **rxode2** (Fidler et al., 2025) compile domain-specific languages for differential equations to C. Statistical modeling languages form another group: Stan (Carpenter et al., 2017) compiles its own probabilistic programming language to C++ with a highly optimized reverse-mode AD library (Carpenter et al., 2015), and is available from R via **rstan**, **cmdstanr** (Gabry et al., 2025), and higher-level interfaces such as **brms** (Bürkner, 2017). Some Stan functions can run on GPUs via OpenCL. **nimble** (de Valpine et al., 2017) generates C++ from a BUGS-like language and from a subset of R, and supports AD via CppAD (Bell, 2012), as do **TMB** (Kristensen et al., 2016) and **RTMB** (Kristensen, 2025), where the latter records the AD tape by evaluating R code with overloaded operators. Forward-mode AD via dual numbers is available in **nabla**, **salad**, **dual**, and **madness** (Towell, 2026; Perdry, 2024; Sartore, 2025; Pav, 2023). Finally, some packages offer GPU acceleration for specific methods, such as **xgboost** (Chen et al., 2025), while **OpenCL** provides a low-level interface for writing kernels by hand. Table 1 summarizes the most closely related packages. To the best of our knowledge, no native R package offers JIT compilation of R code, reverse-mode AD, and GPU execution at the same time.

1.1 Contributions

We introduce **anvl**, a code transformation framework for array programs in R. Users write numerical algorithms in R against a set of array operations, and **anvl** transforms them: into an executable via JIT compilation, into their gradient via reverse-mode AD, or both.

The same program runs on CPUs and NVIDIA GPUs. Transformations are higher-order functions that take an R function and return another R function, similar in spirit to base R's `Vectorize()`, and they compose: the gradient of a function can be called from within a jitted function, which can in turn be called from another one.

The primary backend, `"pjrt"`, targets the OpenXLA compiler ecosystem (OpenXLA Team, 2026), which also powers JAX and TensorFlow. Part of our contribution are the **stablehlo** package, which allows creating programs in the StableHLO language from R, and the **pjrt** package, which compiles and executes them via the PJRT runtime interface. A second, experimental backend translates programs to Fortran via **quickr**.

A focus of **anvl**'s design is control over numerical precision. Accelerators run much faster in single than in double precision, while many statistical computations, such as sums over many log-likelihood terms, require double precision. R, however, stores all real numbers in double precision, so data passed in from R says nothing about the intended precision. In **anvl**, R values – scalars as well as arrays – have no data type until they meet an array, whose data type they then take. Together with scoped default data types and promotion rules that each function declares explicitly, this allows choosing the precision per computation, without silently losing precision when data enters a program.

The package separates *what* an algorithm computes from *how* it is executed. Because the backends do the heavy lifting, **anvl** itself and **stablehlo** are written in R, which makes them comparatively easy to extend with new operations, transformations, or backends. We see **anvl** as low-level infrastructure on which new algorithms and modeling packages can be built, rather than as a modeling package itself. In contrast to **quickr**, which compiles a subset of existing R code, **anvl** defines its own array interface with more data types, explicit devices, and consistent semantics, most of it available through the familiar R generics.

The remainder of this paper is structured as follows. Section 2 introduces the OpenXLA components and the **quickr** package, as well as the basics of reverse-mode AD. Section 3 describes the design of **anvl**, Section 4 its treatment of data types and precision, and Section 5 shows how to extend it. In Section 6, we implement a Hamiltonian Monte Carlo sampler for Bayesian logistic regression, whose runtime we evaluate against Stan in Section 7. Finally, we discuss limitations and future work in Section 8.

2 Background

2.1 The OpenXLA compiler ecosystem

OpenXLA (OpenXLA Team, 2026) is an open-source compiler ecosystem for machine learning that originated in TensorFlow. One of its goals is to decouple array computing frameworks (the frontends) from hardware-specific compilers and runtimes (the backends) through two standardized interfaces:

1. StableHLO (StableHLO Team, 2026), a portable intermediate representation for array programs, and
2. PJRT (PJRT Team, 2026), a runtime interface for compiling and executing such programs on a device.

A framework that emits StableHLO and executes it via PJRT can run on any hardware for which a PJRT plugin exists, including CPUs, NVIDIA GPUs, and Google TPUs. This turns an $n \times m$ problem (n frontends targeting m types of hardware) into an $n + m$ problem. The main compiler consuming StableHLO is XLA, which, among other things, fuses operations to reduce memory traffic, but alternatives such as IREE (The IREE Authors, 2019) exist.

StableHLO

StableHLO is a dialect of MLIR, the Multi-Level Intermediate Representation that is part of the LLVM compiler infrastructure. It is a strongly typed, functional language in static single

assignment form: every value is assigned exactly once, and the type of each value, i.e., its data type and shape, is known when the program is compiled. The specification defines around 100 operations, ranging from elementwise arithmetic to control flow, reductions, and random number generation. The **stablehlo** package provides an S3 representation of the language that closely follows the specification, a `repr()` generic that converts it to the textual format understood by the compiler, type inference functions for the operations, and a builder API. Below, we create a program that multiplies two `f32` matrices of shape `2x2`.

```
library(stablehlo)
func <- local_func("main")
x <- hlo_input("x", "f32", shape = c(2, 2))
y <- hlo_input("y", "f32", shape = c(2, 2))
func <- hlo_return(hlo_multiply(x, y))
func
#> func.func @main (%x: tensor<2x2xf32>, %y: tensor<2x2xf32>) -> tensor<2x2xf32> {
#> %0 = stablehlo.multiply %x, %y : tensor<2x2xf32>
#> return %0 : tensor<2x2xf32>
#> }
```

`local_func()` creates a new function, and `hlo_input()` adds typed inputs to it and returns the corresponding values. Calling `hlo_multiply()` runs the type inference for the operation, which checks that it is valid for the given inputs, and records it in the body of the function. Finally, `hlo_return()` sets the outputs and returns the finished `Func`. The package is written in R rather than wrapping the C++ implementation, which keeps it easy to install. It does not implement the full specification; for example, complex numbers and dynamic shapes are not supported.

PJRT

The PJRT interface defines, among other things, how to allocate buffers on a device, compile a program for it, and execute the compiled program. Its implementations are shared libraries (plugins) that are loaded at runtime, and the **pjrt** package wraps the C API. Below, we compile the program from above and execute it on the CPU.

```
library(pjrt)
program <- pjrt_program(src = repr(func), format = "mlir")
executable <- pjrt_compile(program, device = "cpu")
x <- pjrt_buffer(1:4, dtype = "f32", shape = c(2, 2), device = "cpu")
y <- pjrt_buffer(5:8, dtype = "f32", shape = c(2, 2), device = "cpu")
as_array(pjrt_execute(executable, x, y))
#>      [,1] [,2]
#> [1,]    5   21
#> [2,]   12   32
```

Because **pjrt** accepts any StableHLO program, it can also execute programs that were not created with **stablehlo**, such as programs exported from JAX. Buffers are external pointers to device memory, which R's serialization cannot handle. They can instead be saved and loaded in the safetensors format via the **safetensors** package (Falbel, 2025).

Building a PJRT plugin requires compiling LLVM, MLIR, and XLA, which takes hours, so **pjrt** instead downloads prebuilt plugins when they are first needed, after asking for permission. The NVIDIA libraries required by the CUDA plugin are installed by a companion R package from NVIDIA's redistributable channels, so users need a compatible driver but no local CUDA toolkit. XLA and StableHLO are licensed under Apache 2.0, the NVIDIA libraries under NVIDIA's license terms, and **anvl**, **stablehlo**, and **pjrt** under the MIT license. Because of the downloads, the packages are currently distributed via R-universe rather than CRAN.

2.2 The quickr package

The **quickr** package (Kalinowski, 2026) compiles R functions to Fortran. Unlike **stablehlo**, it does not offer a builder API, but statically analyzes the code of an R function whose argument types are declared via `declare()`. In contrast to XLA, a compiled **quickr** function can be reused for inputs of different shapes, and compilation is fast for small programs. On the other hand, it only runs on the CPU, supports fewer data types, and has no AD. **anvl**'s **quickr** backend is at an early, experimental stage: it covers only part of **anvl**'s operations and does not yet exploit the support for dynamic shapes.

2.3 Reverse-mode automatic differentiation

AD exploits that a program evaluating $F : \mathbb{R}^n \to \mathbb{R}^m$ executes a sequence of elementary operations $v_i = \varphi_i(v_j)_{j \prec i}$ with known derivatives, where $j \prec i$ denotes that v_j is an argument of φ_i (Griewank and Walther, 2008). Forward mode applies the chain rule along this evaluation procedure and computes a Jacobian-vector product $F'(x)\dot{x}$ per sweep. Reverse mode instead propagates *adjoints* $\bar{v}_j = \sum_{i \succ j} \bar{v}_i \frac{\partial v_i}{\partial v_j}$ backward from the outputs and computes a vector-Jacobian product $\bar{y}^\top F'(x)$ per sweep. For a scalar function, a single reverse sweep therefore yields the whole gradient at a small constant multiple of the cost of evaluating F, which makes reverse mode the method of choice for optimization and sampling, at the price of storing intermediate values for the backward sweep.

3 Package design

The central idea of **anvl**, which is adopted from JAX, is to view compilation and differentiation as *code transformations*. There are three kinds of them:

1. R function → graph, via tracing.
2. Graph → graph, e.g., reverse-mode AD.
3. Graph → executable, via one of the backends.

In this section, we first introduce the array type and eager execution, then describe the three kinds of transformations, and finally how they are exposed to users through `jit()` and `gradient()`.

3.1 Arrays and eager execution

The main data structure is the `AnvlArray`. Compared to an R array, it supports more data types (e.g., single and double precision floats and signed and unsigned integers of different widths), lives on a specific device, such as a CPU or a GPU, and represents scalars as arrays without axes. Arrays have value semantics.

```
library(anvl)
x <- nv_array(1:6, shape = c(2, 3), dtype = "f32")
x
#> AnvlArray
#>  1 3 5
#>  2 4 6
#> [ CPUf32{2,3} ]
```

Operations are available through `nv_*()` functions and the corresponding R generics and operators. Called on arrays, they are executed immediately, which is called *eager* execution. Plain R values take the data type of the array they meet, so the literal `2` below becomes an `f32` (see Section 4).

```
sum(x * 2)
#> AnvlArray
#>  42
#> [ CPUf32{} ]
```

Under the hood, there are two layers of functions. *Primitives*, named `prim_*()`, are the operations that the transformations know about, each defined by a set of rules. They are deliberately minimal and, for example, neither broadcast nor promote data types. The user-facing `nv_*()` functions build on the primitives and add conveniences such as broadcasting of scalars, type promotion, and default arguments. Each primitive is itself a jitted function, so eager execution means compiling and caching one small program per primitive and input type. This is convenient for interactive work and debugging, but for performance, the whole program should be compiled, as we describe below.

Execution is asynchronous: calling a compiled program returns as soon as the computation has been scheduled, and the result is only awaited when its values are needed, e.g., when it is printed or converted to R via `as_array()`. Metadata such as `shape()` and `dtype()` is available immediately. This allows R to prepare the next call while the device is still busy, which is especially important for GPUs.

3.2 Tracing R code into a graph

There are two ways to translate code into another representation: statically analyzing its source code, as done by **quickr**, or executing it with special values that record the operations applied to them, which is called tracing. **anvl** follows the latter approach. The function `trace_fn()` evaluates an R function with `GraphBox` objects in place of arrays, which carry only a data type and a shape. Below, we trace a function that either adds or multiplies two scalars, depending on an argument `op`, where `nv_aval()` creates such an abstract value, here an `f32` scalar.

```
f <- function(x, y, op) {
  if (op == "add") x + y else x * y
}
graph <- trace_fn(f, list(x = nv_aval("f32", integer()), y = nv_aval("f32", integer()),
  op = "mul"))
graph
#> <AnvlGraph> (%x1: f32[], %x2: f32[]) {
#>   %1: f32[] = mul(%x1, %x2)
#>   return %1
#> }
```

Every call to a primitive appends a `PrimitiveCall`, consisting of the primitive, its inputs, its static parameters, and its outputs, to the graph under construction. All other R code is simply evaluated. Here, the `if` statement is executed by R, and only the branch that was taken becomes part of the `AnvlGraph`. The graph holds its inputs and outputs, the primitive calls, and the arrays that the traced function closes over. Since graphs only operate on arrays, the nesting structure of (nested) lists of inputs and outputs is stored separately, which allows transformed functions to take and return lists of arrays.

Tracing has two main advantages over static analysis. First, arbitrary R code can be used to construct a program, as long as the operations on arrays are **anvl** operations. Arguments that are not arrays, such as `op` above, are ordinary R values during tracing, which allows reusing R's control flow, functions, and data structures to generate programs. Second, the approach only requires that the array operations are known, and not the semantics of the R language as a whole. The downside is that tracing only works for functions that can be intercepted. For example, `sum()` and `%*%` are generics with methods for **anvl**'s arrays, whereas `rowSums()` is not generic and therefore cannot be traced; a static compiler such as **quickr** can translate such functions directly. Furthermore, programs must be pure: side

effects, such as printing or modifying global variables, only happen once during tracing, and R control flow can only depend on values that are known during tracing, such as static arguments and shapes.

3.3 Transforming a graph into another graph

Once a program is represented as an `AnvlGraph`, it can be transformed into another graph. Currently, the main such transformation is reverse-mode AD, implemented in `transform_gradient()`.

```
grad_graph <- transform_gradient(graph, wrt = c("x", "y"))
```

For this, every primitive stores a `"reverse"` rule, which receives the inputs and outputs of the forward call as well as the adjoints of the outputs, and returns the vector-Jacobian products for the inputs. `transform_gradient()` walks the graph in reverse order, applies these rules, and accumulates the adjoints of values that are used multiple times. Because the reverse rules themselves call primitives, the backward pass is recorded with the same tracing machinery and the result is again an `AnvlGraph`. The transformation is defined for functions with a scalar output, which is the case for the objective functions of optimization and sampling algorithms. **anvl** does not prescribe how a transformation is implemented; the rules attached to primitives are simply a convenient place to store per-operation information.

3.4 Transforming a graph into an executable

A backend lowers an `AnvlGraph` into an executable. For the `"pjrt"` backend, which uses XLA, `stablehlo()` translates the graph into a StableHLO function by applying each primitive's `"stablehlo"` rule, which maps it to one or more **stablehlo** operations.

```
stablehlo(graph)[[1L]]
#> func.func @main (%0: tensor<f32>, %1: tensor<f32>) -> tensor<f32> {
#> %2 = stablehlo.multiply %0, %1 : tensor<f32>
#> return %2 : tensor<f32>
#> }
```

Note that StableHLO uses 0-based indexing, which the rules convert from R's 1-based indexing. R literals are embedded into the program as constants, while arrays that the traced function closes over become additional inputs, which avoids copying their data into the program. The resulting program is then compiled via **pjrt**. The **quickr** backend instead translates the graph into an R function with type declarations and compiles it with **quickr**.

3.5 The user interface

In practice, users rarely call the transformations above themselves, but use `jit()`, `gradient()`, and `value_and_gradient()`, which return functions that apply the transformations lazily, once the types of the inputs are known.

`jit()`

`jit()` returns a function with the same arguments that traces, lowers, and compiles its input when it is called. The compiled executable is cached, and a later call with inputs of the same data types and shapes reuses it. Below, we include a side effect in the function to illustrate this.

```
f_jit <- jit(function(x, y) {
  cat("tracing\n")
```

```
  x * y
})
y1 <- f_jit(x, x)
#> tracing
y2 <- f_jit(x, x + 1)
```

The second call does not print, because it executes the cached executable without tracing the function again. A call with arrays of a different type or shape results in a cache miss. Arguments that are not arrays must be marked as `static`. Their values become part of the cache key, and the function is traced again for every new value.

```
g <- jit(f, static = "op")
as_array(g(x, x, op = "add"))
#>      [,1] [,2] [,3]
#> [1,]    2    6   10
#> [2,]    4    8   12
```

A jitted function called from within another jitted function is not executed separately, but traced as part of the outer function, so that both are compiled into a single executable. Because every primitive is a jitted function, the same function can therefore be used both eagerly and as a building block of larger programs.

`gradient()`

`gradient()` takes a function with scalar output and returns a function that computes the gradient with respect to the arguments given in `wrt`, returning a named list. Like any other function built from **anvl** operations, it can be jitted or called from within another jitted function.

```
loss <- function(beta, X, y) mean((X %*% beta - y)^2)
update <- jit(function(beta, X, y, lr) {
  beta - lr * gradient(loss, wrt = "beta")(beta, X, y)$beta
})
X <- nv_array(matrix(rnorm(20), 10, 2))
y <- X %*% matrix(c(1, 2))
as_array(update(nv_array(matrix(0, 2, 1)), X, y, lr = 0.1))
#>            [,1]
#> [1,] 0.04324131
#> [2,] 0.37689808
```

When `update()` is traced, the call to the gradient function traces `loss()` into a separate graph, transforms it, and inlines the resulting gradient graph into the graph of `update()`.

Control flow

R's control flow can be used within jitted functions, but as we have seen, an `if` statement only retains the branch that was taken and a loop is unrolled during tracing. For control flow that depends on array values, or for loops that should not be unrolled, **anvl** provides the higher-order primitives `nv_if()`, `nv_while()`, and `nv_scan()`, which trace the functions passed to them into sub-graphs. For example, `nv_scan()` runs a loop for a fixed number of steps, threads a state (the carry) through a body function, and stacks the outputs of all steps; we use it in Section 6.

3.6 Restrictions

The XLA compiler requires all shapes to be known when a program is compiled. This allows for aggressive optimizations, but it means that operations whose output shape depends

on the values of their inputs, such as `unique()` or subsetting with a logical mask, cannot be expressed directly. A common workaround is to pad inputs to a fixed size and mask irrelevant entries, e.g., via `nv_ifelse()`, at the cost of some wasted computation. Moreover, each new combination of input shapes triggers a new compilation, which can be mitigated by padding inputs to a few fixed sizes.

A related restriction is that StableHLO programs cannot raise errors at runtime. For example, a dynamic slice clamps out-of-bounds indices to the valid range instead of failing. **anvl** therefore performs all checks during tracing, and values that must be validated, such as subsetting indices whose range matters, need to be static arguments.

4 Data types and precision

Numerical precision is a recurring concern in statistics. Log-likelihoods sum many terms, linear systems can be ill-conditioned, and Markov chain Monte Carlo methods compare densities whose difference is small relative to their size, all of which can require double precision. Accelerators, on the other hand, are designed for single precision: consumer GPUs often execute double-precision operations at a small fraction of the single-precision throughput, and some devices do not support double precision at all. A framework for statistical computing should therefore make it easy to choose the precision of a computation, and it should not lose precision without being asked to. In this section, we describe how **anvl** achieves this, and how its approach differs from JAX, from which much of **anvl**'s design is otherwise adopted.

4.1 R values have no data type

In JAX, Python scalars are *weakly typed*: they take the data type of the array they are combined with, so adding `2.0` to a single-precision array does not promote it (Bradbury et al., 2018). NumPy arrays, however, have a data type. By default, JAX represents values in 32 bits, so double-precision NumPy data is truncated to single precision when it enters a program; with 64-bit mode enabled, it is kept in double precision and promotes every single-precision array it meets. Transferred to R, where every real-valued vector and array is stored in double precision, both options are unsatisfactory: either all data loses precision when it enters a program, or it pulls every computation into double precision.

anvl therefore extends weak typing to R values that enter a program, i.e., length-one vectors as well as matrices and arrays. An R value has no data type, and only *materializes* as an array when it meets one, taking its data type. Below, the same R matrix `B` is multiplied with a single- and a double-precision array.

```
B <- matrix(c(0.1, 0.2, 0.3, 0.4), 2, 2)
matmul <- jit(function(x, b) x %*% b)
dtype(matmul(nv_array(diag(2), dtype = "f32"), B))
#> <f32>
dtype(matmul(nv_array(diag(2), dtype = "f64"), B))
#> <f64>
```

In the second call, `B` enters the program in double precision and the product is exactly `B`, whereas a representation of `B` at a fixed default precision would either have lost precision or turned the first result into `f64`. Only a value that meets no array, such as the argument of `nv_exp(1)`, settles on a default data type.

When an R value is passed to a jitted function as a non-static argument, as `b` above, its data type is not decided at the call boundary. It enters the trace as an `RData` object, which has a shape but no data type, and each operation that uses it decides at which data type it materializes. If a value is used at several data types, it is transferred at the narrowest data type that holds all of them, and the uses convert it within the compiled program. Below, `b` is used in a single-precision matrix product and converted to `f64`; the graph shows that

it is transferred as an `f64` (`<- double`) and converted to `f32` only for the product, so no information is lost for the second output.

```
trace_fn(function(x, b) list(x %*% b, nv_convert(b, "f64")),
  list(x = nv_aval("f32", c(2L, 2L)), b = nv_aval("double", c(2L, 2L))))
#> <AnvlGraph> (%x1: f32[2,2], %x2: f64[2,2] <- double) {
#>   %1: f32[2,2] = convert [dtype = f32] (%x2)
#>   %2: f32[2,2] = dot_general [
#>    contracting_axes = list(2, 1), batching_axes = list(integer(0), integer(0)),
#>      precision = "highest"
#>   ] (%x1, %1)
#>   return (%2, %x2)
#> }
```

The same principle guards against silent changes of values in other places. An R value is built at the data type it takes, rather than first materialized at a default and then converted, so a value that the data type cannot represent is an error: adding `-2L` to an unsigned 8-bit integer fails instead of wrapping around. The distinction between R's integers and doubles is preserved, so, as in R, `x - 1L` keeps an integer array `x` an integer, while `x - 1` gives a float. Finally, integers never become floats on their own: an unsigned and a signed 64-bit integer have no common data type in **anvl** and must be converted explicitly, whereas JAX and NumPy silently promote them to a float.

4.2 Choosing the precision

The default data types for R doubles and integers are a property of the backend (`f32` and `i32` for XLA) and can be changed for a scope via `with_default_dtypes()` or the option `anvl.default_dtypes`. `with_dtypes()` goes one step further and turns a function into one that runs at the given data types, converting its array arguments and results and setting the defaults for its body.

```
dtype(nv_exp(1))
#> <f32>
dtype(with_default_dtypes(c(float = "f64"), nv_exp(1)))
#> <f64>
loss <- function(beta, X, y) mean((X %*% beta - y)^2)
loss_f64 <- with_dtypes(loss, c(float = "f64"))
dtype(loss_f64(nv_array(matrix(0, 2, 1), dtype = "f32"), B, B[, 1, drop = FALSE]))
#> <f64>
```

Compiled programs are cached per default data types, so both variants can be used side by side. In contrast to JAX's 64-bit mode, which decides whether 64-bit types are available at all, the defaults in **anvl** only decide what an R value settles on when nothing else determines its data type; double precision is always available.

4.3 Promotion rules

When a function receives arrays of different data types, it has to decide on the data type of the computation. In **anvl**, this is not a global rule, but each `nv_*()` function states it explicitly, by passing a *promotion rule* to `as_anvl_arrays()`, which converts its arguments to arrays (materializing R values) at the beginning of the function. `promotion_common()` brings all inputs to their common data type, as used, e.g., by `nv_add()`; `promotion_like()` brings them to the data type of one designated input; `promotion_dtype()` to a fixed data type; and `promotion_rdata_common()`, which is used by primitives, only lets R values adapt and requires arrays to agree. The rules refuse conversions that lose precision unless `coerce = TRUE`. For example, the following function computes $ax + y$ at the data type of `y`, and refuses an `x` of higher precision rather than silently rounding it.

```
axpy <- jit(function(a, x, y) {
  args <- as_anvl_arrays(a = a, x = x, y = y, .promote = promotion_like("y"))
  args$a * args$x + args$y
})
y32 <- nv_array(c(1, 1), dtype = "f32")
as_array(axpy(2, nv_array(c(1, 2), dtype = "f32"), y32))
#> [1] 3 5
axpy(2, nv_array(c(1, 2), dtype = "f64"), y32)
#> Error:
#> ! Cannot bring `x` to data type "f32".
#> x "f64" is not promotable to "f32".
#> i Convert it explicitly with `nv_convert()`.
```

The same mechanism is available to developers of packages built on **anvl**, who can decide for each function which argument determines the precision of the computation, e.g., that a likelihood is always accumulated in double precision while the data stays in single precision.

5 Extending the package

One of our design goals is that **anvl** can be extended with new primitives, transformations, and backends.

5.1 New primitives

Most operations can be expressed in terms of existing primitives and are best added as `nv_*()` functions. A new primitive is needed if an operation cannot be expressed otherwise, or if it benefits from a dedicated implementation, e.g., of its derivative. Below, we add a primitive for the softplus function $\log(1 + e^x)$, which appears in the logistic regression example in Section 6. A primitive is created via `new_primitive()`, whose body records a call in the graph via `graph_desc_add()`. The latter requires an inference function that computes the types of the outputs from those of the inputs and validates them. Within the body, `self` refers to the primitive being defined.

```
prim_softplus <- new_primitive("softplus", function(x) {
  infer <- function(x) {
   if (!(dtype(x) %in% c("f32", "f64"))) cli::cli_abort("{.arg x} must be a float.")
    list(AbstractArray(dtype = dtype(x), shape = shape(x)))
  }
  graph_desc_add(self, list(x = x), infer_fn = infer)[[1L]]
})
```

Next, we attach the StableHLO rule, which receives **stablehlo** values and returns a list of them, and a reverse rule, which uses the derivative $\sigma(x) = 1/(1 + e^{-x})$.

```
prim_softplus[["stablehlo"]] <- function(x) {
  list(hlo_log_plus_one(hlo_exponential(x)))
}
prim_softplus[["reverse"]] <- rule_reverse(
  function(inputs, outputs, grads, params, required) {
    list(prim_mul(grads[[1L]], prim_plogis(inputs[[1L]])))
  }
)
```

The primitive can now be used eagerly, within jitted functions, and be differentiated.

```
z <- nv_array(c(-1, 0, 1))
as_array(jit(gradient(function(z) sum(prim_softplus(z))))(z)$z)
#> [1] 0.2689414 0.5000000 0.7310586
```

This mechanism covers every operation that can be expressed in StableHLO. For operations that cannot, such as calls into LAPACK or cuSOLVER for some linear algebra primitives, the XLA backend supports PJRT's custom call mechanism: a handler written in C or C++ against XLA's foreign function interface is registered from R via **pjrt** and referenced from the StableHLO program. Because handlers are registered per platform, a primitive can dispatch to separate CPU and CUDA implementations.

5.2 New transformations and backends

Since an `AnvlGraph` is a simple data structure consisting of inputs, outputs, and a list of primitive calls, new graph-to-graph transformations can be implemented in R by iterating over the calls and emitting new ones, usually guided by a rule stored in each primitive. For example, forward-mode AD would propagate Jacobian-vector products, and a vectorizing transformation similar to JAX's `vmap()` would rewrite each call to operate on an additional batch axis. A backend is represented by an `AnvlBackend` object, a collection of functions that define how arrays are stored, converted to R, and placed on devices, and how a graph is compiled, together with a lowering rule for each supported primitive. Two backends are available, `"pjrt"` and `"quickr"`; exactly one of them is active at a time, which can be changed via `local_backend()` or `with_backend()`.

5.3 Testing

We test the primitives against **torch** as a reference implementation wherever possible: the outputs of jitted primitives and their gradients are compared to those of **torch** and its autograd engine on random inputs. Operations without a counterpart in **torch** are tested against manually computed results.

6 Use case: Hamiltonian Monte Carlo

In this section, we implement Hamiltonian Monte Carlo (HMC) (Neal, 2011) for Bayesian logistic regression. HMC is a Markov chain Monte Carlo method that uses the gradient of the log posterior density to propose distant states that are still accepted with high probability. It is a natural use case for **anvl**: it needs gradients, it is an iterative algorithm whose loops are slow in R, and running many chains in parallel is a good fit for GPUs.

We observe a binary response $y \in \{0,1\}^n$ and features $X \in \mathbb{R}^{n \times p}$, and assume $y_i \sim \text{Bernoulli}(\sigma(x_i^\top \beta))$ with a standard normal prior $\beta \sim \mathcal{N}(0, I_p)$. HMC simulates the dynamics of a particle with position β and momentum $\rho \sim \mathcal{N}(0, I_p)$ in the potential energy

$$U(\beta) = -\log p(\beta \mid X, y) = -\sum_{i=1}^{n} \left(y_i x_i^\top \beta - \log(1 + e^{x_i^\top \beta}) \right) + \frac{1}{2}\|\beta\|^2 + \text{const.}$$

We write the sampler for K chains at once, so the positions are stored in a $K \times p$ matrix, and the potential returns a vector of length K.

```
potential <- function(beta, X, y) {
  eta <- X %*% t(beta)
  y <- nv_broadcast_to(y, shape(eta))
  loglik <- nv_sum(y * eta - nv_log1p(exp(eta)), axes = 1L)
  logprior <- -nv_sum(beta^2, axes = 2L) / 2
  -(loglik + logprior)
```

```
}
```

Because the chains are independent, the gradient of the summed potential contains the gradient of each chain in the corresponding row. We therefore obtain the gradients of all chains with a single call to `gradient()`.

```
grad_potential <- gradient(
  function(beta, X, y) sum(potential(beta, X, y)),
  wrt = "beta"
)
```

The dynamics are simulated with the leapfrog integrator, which alternates half steps of size $\varepsilon/2$ for the momentum with full steps of size ε for the position. We run L such steps with `nv_scan()`, where the gradient at the current position is part of the carry, so that each step evaluates the gradient only once.

```
leapfrog <- function(beta, p, X, y, eps, L) {
  init <- list(beta = beta, p = p, grad = grad_potential(beta, X, y)$beta)
  nv_scan(init, steps = L, body = function(s, x) {
    p <- s$p - eps / 2 * s$grad
    beta <- s$beta + eps * p
    grad <- grad_potential(beta, X, y)$beta
    p <- p - eps / 2 * grad
    list(carry = list(beta = beta, p = p, grad = grad), out = NULL)
  })$carry
}
```

A single transition draws a new momentum, simulates a trajectory, and accepts the proposal of each chain with probability $\min(1, \exp(H_0 - H_1))$, where $H = U(\beta) + \|\rho\|^2/2$ is the total energy. The random number generator state is passed explicitly, as **anvl** is functional, and the acceptance decision uses `nv_ifelse()` instead of an R `if` statement, as it depends on array values.

```
hmc_step <- function(beta, state, X, y, eps, L) {
  draw <- nv_rnorm(shape(beta), state)
  p <- draw$values
  prop <- leapfrog(beta, p, X, y, eps, L)
  H0 <- potential(beta, X, y) + nv_sum(p^2, axes = 2L) / 2
  H1 <- potential(prop$beta, X, y) + nv_sum(prop$p^2, axes = 2L) / 2
  u <- nv_runif(shape(beta)[1L], draw$state)
  accept <- log(u$values) < H0 - H1
  accept <- nv_broadcast_to(accept, shape(beta))
  list(beta = nv_ifelse(accept, prop$beta, beta), state = u$state)
}
```

Finally, we collect `n_draws` draws with another `nv_scan()`, whose outputs are stacked along a new first axis, and compile the whole sampler with `jit()`. The number of leapfrog steps and draws are static arguments, as they determine the shapes and structure of the program.

```
hmc <- jit(function(beta, state, X, y, eps, L, n_draws) {
  out <- nv_scan(list(beta = beta, state = state), steps = n_draws,
    body = function(s, x) {
      s <- hmc_step(s$beta, s$state, X, y, eps, L)
      list(carry = s, out = s$beta)
    }
```

```
  )
  list(draws = out$out, beta = out$carry$beta, state = out$carry$state)
}, static = c("L", "n_draws"))
```

We apply the sampler to simulated data with $n = 1000$ observations and $p = 5$ features, running four chains in double precision.

```
options(anvl.default_dtypes = c(float = "f64", int = "i32"))
set.seed(1)
n <- 1000; p <- 5
X <- matrix(rnorm(n * p), n, p)
y <- rbinom(n, 1, plogis(X %*% rnorm(p, sd = 0.5)))
res <- hmc(nv_array(matrix(0, 4, p)), nv_rng_state(42L), nv_array(X),
  nv_array(as.double(y)), eps = 0.5 / sqrt(n), L = 10L, n_draws = 2000L)
draws <- as_array(res$draws)
dim(draws)
#> [1] 2000    4    5
```

The posterior means, computed from the second half of each chain, agree with the maximum likelihood estimates up to the shrinkage induced by the prior.

```
rbind(
  hmc = colMeans(apply(draws[1001:2000, , ], 3, c)),
  glm = coef(glm(y ~ X - 1, family = binomial))
)
#>             X1        X2         X3        X4        X5
#> hmc -0.7730560 0.3412134 -0.8296505 0.6687511 0.5593170
#> glm -0.7738017 0.3425455 -0.8321399 0.6701697 0.5599573
```

The whole sampler, including the 2000×11 gradient evaluations per chain, runs as a single compiled program without returning to R. The same code runs on a GPU when the arrays are allocated there, e.g., via `nv_array(X, device = "cuda")`.

7 Runtime evaluation

In this section, we compare the runtime of the HMC sampler from Section 6 to Stan, whose reverse-mode AD library and code generation are the state of the art for statistical models on the CPU (Carpenter et al., 2015). We ask how the sampler compares on a single CPU core, how well it uses multiple cores and a GPU, both within one chain and across many chains, and how large the compilation overhead is.

7.1 Setup

We run Stan's static HMC sampler (without the No-U-Turn extension) via CmdStan 2.40, with the same step size $\varepsilon = 0.5/\sqrt{n}$, $L = 10$ leapfrog steps, the identity metric, and adaptation disabled, so that both samplers perform the same number of gradient evaluations. The Stan model uses `bernoulli_logit_glm()`, Stan's specialized and most efficient implementation of the logistic regression likelihood. Stan can also evaluate this likelihood on a GPU via OpenCL, which we did not use. Both samplers work in double precision and start at $\beta = 0$, and the data are simulated with standard normal features and $\beta \sim \mathcal{N}(0, 0.25 I_p)$. We exclude process start-up and data loading from Stan's measurements by subtracting the time of an otherwise identical run without draws, and time **anvl** after the data has been transferred to the device. Each configuration is repeated three times after a first call that includes compilation, and we report the median.

We run three experiments. First, we draw 1000 samples from a single chain for $p = 20$ and $n \in \{10^3, 10^4, 10^5\}$. Second, we measure strong scaling: we draw 200 samples from a

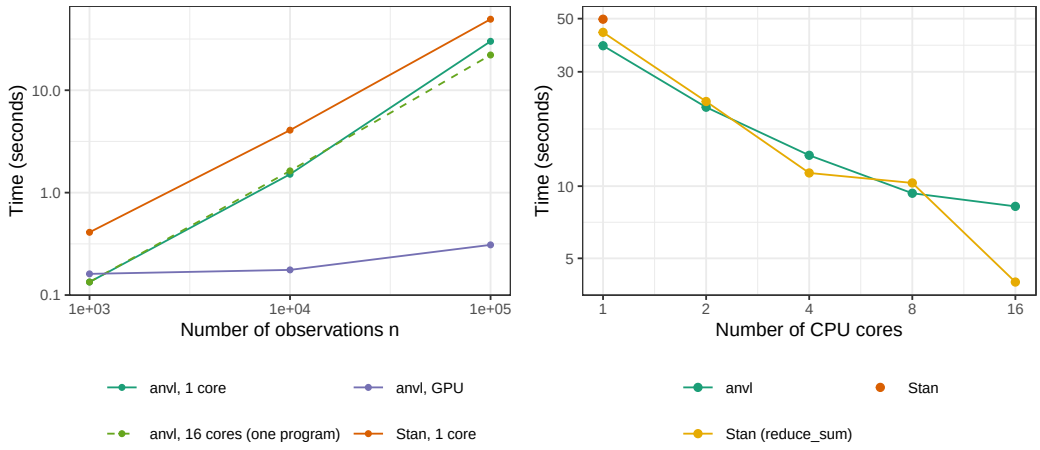


Figure 1: Left: time to draw 1000 samples from a single chain as a function of the number of observations n ($p = 20$). Right: strong scaling, time to draw 200 samples from a single chain for $n = 10^5$ and $p = 100$ on 1 to 16 CPU cores; Stan uses reduce_sum(), the single point is the unmodified Stan model, and the dashed line is **anvl** on the GPU. Medians of three repetitions, log-log scales.

single chain for $n = 10^5$ and $p = 100$ on 1 to 16 cores. **anvl** runs the unchanged sampler, which XLA parallelizes within operations. Stan needs a modified model for this, in which `reduce_sum()` splits the likelihood into slices that are evaluated by several threads. Third, we draw 1000 samples from each of K chains for $n = 10^4$ and $p = 20$. Static HMC in CmdStan runs one chain per process, so for K chains we start K processes, of which at most as many run at the same time as cores are available. On one core and on the GPU, **anvl** runs all chains as one batched program. On 16 cores, we split the chains into 16 groups and run each group on its own PJRT CPU device (via the environment variable `PJRT_CPU_DEVICE_COUNT`), so that, as for CmdStan, each core works on its own chains; we also report the time of a single program for all chains.

All experiments ran on the LRZ AI Systems in the Docker image described in Appendix A. The CPU experiments used a node with two Intel Xeon Gold 6336Y processors, restricted to 16 physical cores, where configurations with fewer cores were pinned to a subset of them, and the GPU experiments used an NVIDIA A100 80GB GPU.

7.2 Results

Figure 1 (left) shows the results for a single chain. On one core, **anvl** draws the samples 3.1, 2.7, and 1.6 times faster than CmdStan for $n = 10^3$, 10^4, and 10^5, respectively. A plausible reason is that XLA compiles the whole sampler, including the leapfrog loop and the gradient, into a single optimized program, whereas Stan evaluates the gradient with its operator-overloading AD library. The GPU does not pay off for small data, as its per-step overhead makes it slower than a CPU core for $n = 10^3$. For $n = 10^5$, however, it draws the samples in 0.31 seconds, which is about 100 times faster than **anvl** on one core and 160 times faster than CmdStan.

Figure 1 (right) shows how a single chain scales with the number of cores. Without any change to the code, **anvl** becomes 1.8, 2.9, 4.1, and 4.7 times faster on 2, 4, 8, and 16 cores. With `reduce_sum()`, Stan scales further at 16 cores (11 times faster than on one thread), but this requires rewriting the model, and the parallelized model is slower than the original on a single core (44 versus 50 seconds, compared to 39 seconds for **anvl**). Up to 8 cores, both scale similarly; beyond that, profiling suggests that **anvl** is limited by the fixed cost of distributing each operation of the gradient over the cores, as the individual operations are too small at this problem size to keep 16 cores busy. The GPU completes the same run in – seconds.

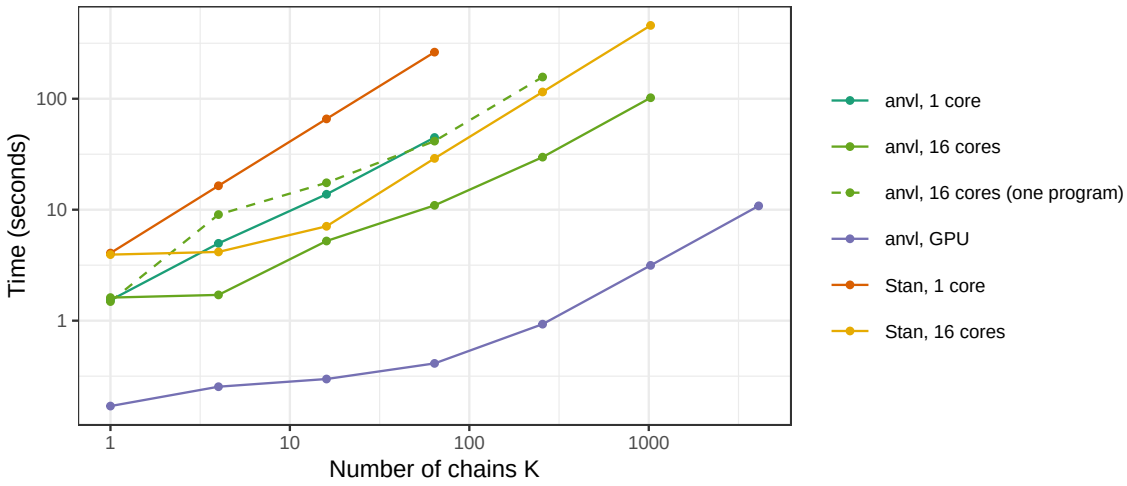


Figure 2: Time to draw 1000 samples from each of K chains for $n = 10^4$ (median of three repetitions, log-log scale).

Figure 2 shows the results for multiple chains. On one core, both samplers scale approximately linearly in the number of chains, with **anvl** being 2.7 (one chain) to 5.9 (64 chains) times faster, as batching the chains turns the matrix-vector products of the individual chains into matrix-matrix products. On 16 cores, both run the chains in parallel, and **anvl** is 2.7 times faster than CmdStan for 64 chains and 4.5 times faster for 1024 chains (102 versus 458 seconds). Running all chains as a single program on 16 cores is considerably slower than splitting them over devices, because each operation is again too small to be distributed efficiently. On the GPU, the time is almost constant up to 64 chains and grows sublinearly beyond that: 4096 chains with 1000 draws each take 10.8 seconds, which corresponds to about 380,000 draws per second, compared to about 2,200 draws per second for CmdStan on 16 cores. This is exactly the regime in which the many-chain MCMC methods discussed in the introduction operate. All configurations of both samplers had an average acceptance rate between 0.98 and 0.99.

Finally, we measured the one-off costs for $n = 10^4$ on 16 cores. Tracing the sampler, lowering it to StableHLO, and compiling it with XLA take 0.28, 0.21, and 0.24 seconds, respectively, so the first call is about half a second slower than later calls (2.1 versus 1.5 seconds). Compiling the Stan model takes about 11 seconds, and **quickr** compiles the gradient in 1.2 seconds, compared to 0.11 seconds for XLA. Although compilation must be repeated for every new combination of input shapes and static arguments, its cost is small compared to the runtime of expensive methods and quickly amortized as the size of the data grows.

8 Discussion

In this paper, we introduced **anvl**, a code transformation framework for array programs in R. Programs are traced into a computational graph, which can be transformed into another graph, as done for reverse-mode AD, and into an executable, either via the OpenXLA ecosystem using **stablehlo** and **pjrt**, or via **quickr**. The XLA backend allows running the same code on CPUs and GPUs. Unlike JAX, **anvl** treats all R values, including arrays, as values without a data type that take the precision of the arrays they meet, and lets every function declare its promotion rule, which gives users control over precision without losing it silently when data enters a program. We demonstrated the package by implementing an HMC sampler as a single compiled program.

The main limitations of the current approach stem from XLA's requirement of static shapes. Operations whose output shape depends on data are not supported, and each new combination of input shapes requires recompilation. Programs must be written in terms

of **anvl**'s operations, and while most of them are accessible through R's generics, tracing cannot intercept functions that are not generic. Furthermore, AD is currently restricted to first-order gradients of scalar functions in reverse mode.

Future work will address these limitations. Adding forward-mode AD would make it possible to compute Jacobians and, combined with reverse mode, Hessian-vector products and Hessians, which are needed, e.g., for Laplace approximations. A vectorizing transformation similar to JAX's `vmap()` would remove the need to write batched code by hand, as we did for the HMC sampler. To lift the static shape restriction, the intermediate representation needs to support dynamic shapes; the **quickr** backend, which already compiles shape-polymorphic code, and the IREE compiler (The IREE Authors, 2019), which also consumes StableHLO, are two possible routes. Beyond the core, we hope that **anvl** will serve as a foundation for packages that implement statistical and machine learning algorithms on top of it.

Acknowledgments

Sebastian Fischer is supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – 460135501 (NFDI project MaRDI). We thank the developers of JAX, whose design and documentation, in particular the Autodidax tutorial (The JAX Authors, 2024), inspired many aspects of **anvl**, as well as the authors of the Go library gopjrt (Pfeifer, 2026), which informed the design of **pjrt**. We gratefully acknowledge the computational resources provided by the Leibniz Supercomputing Centre (LRZ).

Appendix

A Reproducibility

All files required to reproduce the results are in the `paper/` directory of the GitHub repository `https://github.com/r-xla/anvl`, which contains a README with instructions. The Dockerfile in `paper/docker/` builds the image in which all results were produced. It is based on the `anvl-cuda` image, pinned by digest, and installs the r-xla packages, **quickr**, and CmdStan at fixed versions. The benchmark scripts are in `paper/benchmarks/`, including the Slurm scripts used to run them on the LRZ AI Systems via Enroot, which imports the Docker image.

B AI usage

AI coding agents were used in the development of the software and the preparation of the manuscript. All AI-generated content was reviewed by the authors.

References

O. Abril-Pla, V. Andreani, C. Carroll, L. Dong, C. J. Fonnesbeck, M. Kochurov, R. Kumar, J. Lao, C. C. Luhmann, O. A. Martin, M. Osthege, R. Vieira, T. Wiecki, and R. Zinkov. PyMC: A modern, and comprehensive probabilistic programming framework in Python. *PeerJ Computer Science*, 9:e1516, 2023. doi: 10.7717/peerj-cs.1516. [p1]

B. M. Bell. Cppad: a package for c++ algorithmic differentiation. *Computational Infrastructure for Operations Research*, 57(10):3, 2012. [p2]

J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. Jax: Composable transformations of python+numpy programs. Github repository. URL `https://github.com/jax-ml/jax`, 2018. [p1, 9]

P.-C. Bürkner. brms: An r package for bayesian multilevel models using stan. *Journal of statistical software*, 80:1–28, 2017. [p2]

A. Cabezas, A. Corenflos, J. Lao, R. Louäf, et al. BlackJAX: Composable Bayesian inference in JAX. *arXiv preprint arXiv:2402.10797*, 2024. [p1]

B. Carpenter, M. D. Hoffman, M. Brubaker, D. Lee, P. Li, and M. Betancourt. The Stan math library: Reverse-mode automatic differentiation in C++. *arXiv preprint arXiv:1509.07164*, 2015. [p2, 14]

B. Carpenter, A. Gelman, M. D. Hoffman, D. Lee, B. Goodrich, M. Betancourt, M. Brubaker, J. Guo, P. Li, and A. Riddell. Stan: A probabilistic programming language. *Journal of statistical software*, 76:1–32, 2017. [p2]

T. Chen, T. He, M. Benesty, V. Khotilovich, Y. Tang, H. Cho, K. Chen, R. Mitchell, I. Cano, T. Zhou, M. Li, J. Xie, M. Lin, Y. Geng, Y. Li, J. Yuan, and D. Cortes. *xgboost: Extreme Gradient Boosting*, 2025. URL https://CRAN.R-project.org/package=xgboost. R package version 3.1.2.1. [p2]

P. de Valpine, D. Turek, C. J. Paciorek, C. Anderson-Bergman, D. T. Lang, and R. Bodik. Programming with models: Writing statistical algorithms for general model structures with NIMBLE. *Journal of Computational and Graphical Statistics*, 26(2):403–413, 2017. doi: 10.1080/10618600.2016.1172487. [p2]

J. V. Dillon, I. Langmore, D. Tran, E. Brevdo, S. Vasudevan, D. Moore, B. Patton, A. Alemi, M. Hoffman, and R. A. Saurous. TensorFlow distributions. *arXiv preprint arXiv:1711.10604*, 2017. [p1]

D. Eddelbuettel and R. François. Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, 40(8):1–18, 2011. doi: 10.18637/jss.v040.i08. [p1]

D. Falbel. *safetensors: Safetensors File Format*, 2025. URL https://CRAN.R-project.org/package=safetensors. R package version 0.2.0. [p4]

D. Falbel and J. Luraschi. *torch: Tensors and Neural Networks with 'GPU' Acceleration*, 2025. URL https://CRAN.R-project.org/package=torch. R package version 0.16.3. [p1]

M. Fidler, M. Hallow, J. Wilkins, and W. Wang. *rxode2: Facilities for Simulating from ODE-Based Models*, 2025. URL https://CRAN.R-project.org/package=rxode2. R package version 5.0.1. [p2]

R. FitzJohn. *odin: ODE Generation and Integration*, 2025. URL https://CRAN.R-project.org/package=odin. R package version 1.2.7. [p2]

R. Frostig, M. J. Johnson, and C. Leary. Compiling machine learning programs via high-level tracing. In *Proceedings of the Conference on Systems and Machine Learning (SysML)*, 2018. [p1]

J. Gabry, R. Češnovar, A. Johnson, and S. Bronder. *cmdstanr: R Interface to 'CmdStan'*, 2025. URL https://mc-stan.org/cmdstanr/. R package version 0.9.0. [p2]

N. Golding. greta: Simple and scalable statistical modelling in R. *Journal of Open Source Software*, 4(40):1601, 2019. doi: 10.21105/joss.01601. [p2]

A. Griewank and A. Walther. *Evaluating derivatives: principles and techniques of algorithmic differentiation*. SIAM, 2nd edition, 2008. doi: 10.1137/1.9780898717761. [p5]

C. R. Harris, K. J. Millman, S. J. Van Der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020. [p1]

M. D. Hoffman and P. Sountsov. Tuning-free generalized Hamiltonian Monte Carlo. In *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics*, volume 151 of *Proceedings of Machine Learning Research*, pages 7799–7813, 2022. [p2]

M. D. Hoffman, A. Radul, and P. Sountsov. An adaptive-MCMC scheme for setting trajectory lengths in Hamiltonian Monte Carlo. In *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, volume 130 of *Proceedings of Machine Learning Research*, pages 3907–3915, 2021. [p2]

T. Kalinowski. *quickr: Compiler for R*, 2026. URL https://github.com/t-kalinowski/quickr. R package version 0.3.0.9000. [p2, 5]

K. Krämer and F. Xie. *ast2ast*, 2026. URL https://github.com/Konrad1991/ast2ast/. [p2]

K. Kristensen. *RTMB: R Bindings for TMB*, 2025. URL https://CRAN.R-project.org/package=RTMB. [p2]

K. Kristensen, A. Nielsen, C. W. Berg, H. Skaug, and B. M. Bell. Tmb: Automatic differentiation and laplace approximation. *Journal of Statistical Software*, 70(5), 2016. ISSN 1548-7660. doi: 10.18637/jss.v070.i05. URL http://dx.doi.org/10.18637/jss.v070.i05. [p2]

C. C. Margossian, M. D. Hoffman, P. Sountsov, L. Riou-Durand, A. Vehtari, and A. Gelman. Nested $\hat{R}$: Assessing the convergence of Markov chain Monte Carlo when running many short chains. *Bayesian Analysis*, 2024. [p2]

F. Morandat, B. Hill, L. Osvald, and J. Vitek. Evaluating the design of the r language: Objects and functions for data analysis. In *European conference on object-oriented programming*, pages 104–131. Springer, 2012. [p2]

R. M. Neal. MCMC using Hamiltonian dynamics. In S. Brooks, A. Gelman, G. L. Jones, and X.-L. Meng, editors, *Handbook of Markov Chain Monte Carlo*, pages 113–162. Chapman and Hall/CRC, 2011. [p12]

OpenXLA Team. *OpenXLA*, 2026. URL https://openxla.org/. [p3]

A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019. [p1]

S. E. Pav. *madness: Automatic Differentiation of Multivariate Operations*, 2023. URL https://github.com/shabbychef/madness. R package version 0.2.8. [p2]

H. Perdry. *salad: Simple Automatic Differentiation*, 2024. URL https://CRAN.R-project.org/package=salad. R package version 1.2. [p2]

J. Pfeifer. *gopjrt: Go Wrappers for OpenXLA PJRT*, 2026. URL https://github.com/gomlx/gopjrt. [p17]

D. Phan, N. Pradhan, and M. Jankowiak. Composable effects for flexible and accelerated probabilistic programming in NumPyro. *arXiv preprint arXiv:1912.11554*, 2019. [p1]

PJRT Team. *PJRT: Pluggable JIT RunTime*, 2026. URL https://openxla.org/pjrt/. [p3]

R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2025. URL https://www.R-project.org/. [p2]

L. Sartore. *dual: Automatic Differentiation with Dual Numbers*, 2025. URL https://CRAN.R-project.org/package=dual. R package version 0.0.6. [p2]

StableHLO Team. *StableHLO*, 2026. URL https://openxla.org/stablehlo/spec. [p3]

The IREE Authors. IREE, Sept. 2019. URL https://github.com/iree-org/iree. [p3, 17]

The JAX Authors. Autodidax: JAX core from scratch, 2024. URL https://docs.jax.dev/en/latest/autodidax.html. [p17]

A. Towell. *nabla: Exact Derivatives via Automatic Differentiation*, 2026. URL https://github.com/queelius/nabla. R package version 0.7.1. [p2]

Sebastian Fischer
LMU MunichMunich Center for Machine Learning (MCML)
Department of Statistics
Ludwig-Maximilians-Universität München
Ludwigstr. 33, 80539 Munich, Germany
ORCiD: 0000-0002-9609-3197
sebastian.fischer@stat.uni-muenchen.de

Daniel Falbel
Posit Software, PBC

ORCiD: 0009-0006-0143-2392

Tomasz Kalinowski
Posit Software, PBC

Nikolai German
LMU Munich

ORCiD: 0009-0001-7394-8367

Martin Binder
LMU MunichMunich Center for Machine Learning (MCML)

ORCiD: 0009-0008-2578-2869

Bernd Bischl
LMU MunichMunich Center for Machine Learning (MCML)

ORCiD: 0000-0001-6002-6980